Assignment 12

Ellen Bledsoe

2026-11-10

Assignment Details

Purpose

The goal of this assignment is to practice writing and using conditional statements, `if_else` and `case_when`, and `if-else` statements, and to combine them with skills from earlier in the semester (joins, strings, and plotting).

Task

Write R code to successfully answer each question below.

Criteria for Success

- Code is within the provided code chunks or new code chunks are created where necessary
- Code chunks run without errors
- Code chunks have brief comments indicating which code is answering which part of the question
- Code will be assessed as follows:
  - Produces the correct answer using the requested approach: 100%
  - Generally uses the right approach, but a minor mistake results in an incorrect answer: 90%
  - Attempts to solve the problem and makes some progress using the core concept, but returns the wrong answer and does not demonstrate comfort with the core concept: 50%
  - Answer demonstrates a lack of understanding of the core concept: 0%
- Any questions requiring written answers are answered with sufficient detail

Due Date

November 18 before class

Assignment Exercises

1. Choice Operators (10 pts)

Create the following variables by running the code chunk below.

```
w <- 10.2
x <- 1.3
y <- 2.8
z <- 17.5
colors <- c("red", "blue", "green")
masses <- c(45.2, 36.1, 27.8, 81.6, 42.4)
```

Using the variables above, write code that determines whether the following statements are `TRUE` or `FALSE`.

a. `"green"` is in `colors`
b. `"purple"` is not in `colors`
c. Each value in `masses` is greater than 40.
d. `w` is greater than `x`, or `y` is greater than `z`
e. `x` times `w` is between 13.2 and 13.5

```
# 1a
"green" %in% colors
```

```
[1] TRUE
```

```
# 1b
!("purple" %in% colors)
```

```
[1] TRUE
```

```
# 1c
masses > 40
```

```
[1]  TRUE FALSE FALSE  TRUE  TRUE
```

```
# 1d
w > x | y > z
```

```
[1] TRUE
```

```
# 1e
x * w > 13.2 & x * w < 13.5
```

```
[1] TRUE
```

2. Using `dplyr` Choice Functions (20 points)

For this question, we will be using functions from `dplyr` (part of the `tidyverse`) and data from the `lterdatasampler` package, so make sure you load both of those packages with the `library()` function.

```
library(dplyr)
library(lterdatasampler)
```

First, create an object called `crab_size` and assign the value 17 to it.

```
crab_size <- 17
```

a. Use the `if_else` function from `dplyr` to say that a crab that is larger than or equal to 20 is "large" and a crab that is smaller than 20 is "small."

```
# 2a
if_else(crab_size >= 20, "large", "small")
```

```
[1] "small"
```

b. Use the `case_when` function to say that if a crab is greater than or equal to 20, it is "large;" if greater than or equal to 10 but less than 20, it is "medium;" and if it is smaller than 10, it is "small." Make sure that your `crab_size` object is still set to 17.

```
# 2b
case_when(crab_size >= 20 ~ "large",
          crab_size >= 10 ~ "medium",
          .default = "small")
```

```
[1] "medium"
```

c. Take a look at the `pie_crab` dataset from the `lterdatasampler` package by running `head(pie_crab)`. We want to create a new column based on the `size` column. Using the `if_else` function inside of a `mutate` function, create a new column called `size_category` based on the same conditions as (a).

I've placed my new column next to the size column for the sake of the answer key; you don't need to do this.

```
# 2c
head(pie_crab)
```

```
# A tibble: 6 x 9
  date       latitude site   size air_temp air_temp_sd water_temp water_temp_sd
  <date>        <dbl> <chr> <dbl>    <dbl>       <dbl>      <dbl>         <dbl>
1 2016-07-24       30 GTM    12.4     21.8        6.39       24.5          6.12
2 2016-07-24       30 GTM    14.2     21.8        6.39       24.5          6.12
3 2016-07-24       30 GTM    14.5     21.8        6.39       24.5          6.12
4 2016-07-24       30 GTM    12.9     21.8        6.39       24.5          6.12
5 2016-07-24       30 GTM    12.4     21.8        6.39       24.5          6.12
6 2016-07-24       30 GTM    13.0     21.8        6.39       24.5          6.12
# i 1 more variable: name <chr>
```

```
pie_crab %>%
  mutate(size_category = if_else(size >= 20, "large", "small"), .after = size)
```

```
# A tibble: 392 x 10
   date       latitude site   size size_category air_temp air_temp_sd water_temp
   <date>        <dbl> <chr> <dbl> <chr>             <dbl>       <dbl>      <dbl>
 1 2016-07-24       30 GTM    12.4 small              21.8        6.39       24.5
 2 2016-07-24       30 GTM    14.2 small              21.8        6.39       24.5
 3 2016-07-24       30 GTM    14.5 small              21.8        6.39       24.5
 4 2016-07-24       30 GTM    12.9 small              21.8        6.39       24.5
 5 2016-07-24       30 GTM    12.4 small              21.8        6.39       24.5
 6 2016-07-24       30 GTM    13.0 small              21.8        6.39       24.5
 7 2016-07-24       30 GTM    10.3 small              21.8        6.39       24.5
 8 2016-07-24       30 GTM    11.2 small              21.8        6.39       24.5
 9 2016-07-24       30 GTM    12.7 small              21.8        6.39       24.5
10 2016-07-24       30 GTM    14.6 small              21.8        6.39       24.5
# i 382 more rows
# i 2 more variables: water_temp_sd <dbl>, name <chr>
```

d. Now use the `case_when` function inside a `mutate` function to create a new column called `size_category3` in the `pie_crab` data frame that meets the same conditions as (b).

I've placed my new column next to the size column for the sake of the answer key; you don't need to do this.

```
# 2d
pie_crab %>%
  mutate(size_category3 = case_when(size >= 20 ~ "large",
                                    size >= 10 ~ "medium",
                                    .default = "small"), .after = size)
```

```
# A tibble: 392 x 10
   date       latitude site   size size_category3 air_temp air_temp_sd
   <date>        <dbl> <chr> <dbl> <chr>              <dbl>       <dbl>
 1 2016-07-24       30 GTM    12.4 medium              21.8        6.39
 2 2016-07-24       30 GTM    14.2 medium              21.8        6.39
 3 2016-07-24       30 GTM    14.5 medium              21.8        6.39
 4 2016-07-24       30 GTM    12.9 medium              21.8        6.39
 5 2016-07-24       30 GTM    12.4 medium              21.8        6.39
 6 2016-07-24       30 GTM    13.0 medium              21.8        6.39
 7 2016-07-24       30 GTM    10.3 medium              21.8        6.39
 8 2016-07-24       30 GTM    11.2 medium              21.8        6.39
 9 2016-07-24       30 GTM    12.7 medium              21.8        6.39
10 2016-07-24       30 GTM    14.6 medium              21.8        6.39
# i 382 more rows
# i 3 more variables: water_temp <dbl>, water_temp_sd <dbl>, name <chr>
```

3. If/Else Statements (20 points)

a. Complete the following `if` statement so that if `age_class` is equal to "sapling" it sets `y` to 10, if `age_class` is equal to "seedling" it sets `y` to 5, and if `age_class` is something else it sets `y` to 0.

```
# age_class <- "adult"
# if (){
#
# }
# y
```

```r
# 3a
age_class <- "adult"
if (age_class == "sapling"){
  y <- 10
} else if (age_class == "seedling") {
  y <- 5
} else {
  y <- 0
}
y
```

```
[1] 0
```

b. Convert your conditional statement from (a) into a function that takes `age_class` as an argument and returns `y`. Call this function 5 times, once with each of the following values for `age_class`: "sapling", "seedling", "adult", "mature", "established".

```r
# 3b
get_y_from_age_class <- function(age_class){
  if (age_class == "sapling"){
    y <- 10
  } else if (age_class == "seedling") {
    y <- 5
  } else {
    y <- 0
  }
  return(y)
}

get_y_from_age_class("sapling")
```

```
[1] 10
```

```r
get_y_from_age_class("seedling")
```

```
[1] 5
```

```r
get_y_from_age_class("adult")
```

```
[1] 0
```

```
get_y_from_age_class("mature")
```

```
[1] 0
```

```
get_y_from_age_class("established")
```

```
[1] 0
```

c. Write a function called `prediction` that takes a single argument `x`. If `x` is greater than 0 and less than 15, return `y = 6 + 0.8 * x`. If `x` is greater than or equal to 15 and less than 30, return `y = 5 + 0.75 * x`. In all other cases, return `y = NA`. Call the function three times, with `5`, `26`, and `-2` as the input.

```
# 3c
prediction <- function(x){
  if (x > 0 & x < 15){
    y <- 6 + 0.8 * x
  } else if (x >= 15 & x < 30) {
    y <- 5 + 0.75 * x
  } else {
    y <- NA
  }
  return(y)
}

prediction(5)
```

```
[1] 10
```

```
prediction(26)
```

```
[1] 24.5
```

```
prediction(-2)
```

```
[1] NA
```

4. Size Estimates by Name (20 points)

This is a follow up to "Use and Modify" from Assignment 11.

To make it even easier to work with your dinosaur size estimation functions you decide to create a function that lets you specify which dinosaur group you need to estimate the size of by name and then have the function automatically choose the right parameters.

Remember the general form of the equation is:

mass <- a * length ^ b

Create a new function `get_mass_from_length_by_name()` that takes two arguments: the `length` and the name of the dinosaur group.

Inside this function use `if`/`else if`/`else` statements to check to see if the name is one of the following values and if so use the associated `a` and `b` values to estimate the species mass using these equations:

- *Stegosauria*: `mass = 10.95 * length ^ 2.64` (Seebacher 2001)
- *Theropoda*: `mass = 0.73 * length ^ 3.63` (Seebacher 2001)
- *Sauropoda*: `mass = 214.44 * length ^ 1.46` (Seebacher 2001)

If the name is not any of these values the function should return `NA`.

```
get_mass_from_length_by_name <- function(length, name){
  if (name == "Stegosauria"){
    a <- 10.95
    b <- 2.64
  } else if (name == "Theropoda"){
    a <- 0.73
    b <- 3.63
  } else if (name == "Sauropoda"){
    a <- 214.44
    b <- 1.46
  } else {
    return(NA)
  }
  mass <- a * length ^ b
  return(mass)
}
```

Run the function for the following:

a. A *Stegosauria* that is 10 meters long.
b. A *Theropoda* that is 8 meters long.

c. A *Sauropoda* that is 12 meters long.
d. An *Ankylosauria* that is 13 meters long.

```
# 4a
get_mass_from_length_by_name(10, "Stegosauria")
```

```
[1] 4779.848
```

```
# 4b
get_mass_from_length_by_name(8, "Theropoda")
```

```
[1] 1385.286
```

```
# 4c
get_mass_from_length_by_name(12, "Sauropoda")
```

```
[1] 8070.685
```

```
# 4d
get_mass_from_length_by_name(13, "Ankylosauria")
```

```
[1] NA
```

Challenge 1 (optional): If the name is not one of values that have a and b values print warning that it doesn't know how to convert that group that includes that groups name in a message like "No known estimation for Ankylosauria." You can use the function `warning("your warning text")` to print a warning and the function `paste()` to combine text with a value from a variable such as `paste("My name is", name)`. Doing this successfully will modify your answer to (d), which is fine.

```
# Challenge 1
get_mass_from_length_by_name <- function(length, name){
  if (name == "Stegosauria"){
    a <- 10.95
    b <- 2.64
  } else if (name == "Theropoda"){
    a <- 0.73
    b <- 3.63
  } else if (name == "Sauropoda"){
```

```
    a <- 214.44
    b <- 1.46
  } else {
    warning(paste("No known estimation for", name))
    return(NA)
  }
  mass <- a * length ^ b
  return(mass)
}

get_mass_from_length_by_name(13, "Ankylosauria")
```

```
Warning in get_mass_from_length_by_name(13, "Ankylosauria"): No known
estimation for Ankylosauria
```

```
[1] NA
```

Challenge 2 (optional): Change your function so that it uses two different values of `a` and `b` for *Stegosauria*. When *Stegosauria* is greater than 8 meters long use the equation above. When it is 8 meters long or less, use `a = 8.5` and `b = 2.8`. Run the function for a *Stegosauria* that is 6 meters long.

Hint: you can put an `if-else` statement inside another `if` statement, like this:

```
if (name == "Stegosauria") {
  if (length > 8) {
    # one set of a and b values
  } else {
    # another set of a and b values
  }
} else if ...
```

```
# Challenge 2
get_mass_from_length_by_name <- function(length, name){
  if (name == "Stegosauria"){
    # nested if-else: equation depends on length
    if (length > 8){
      a <- 10.95
      b <- 2.64
    } else {
      a <- 8.5
```

```
      b <- 2.8
    }
  } else if (name == "Theropoda"){
    a <- 0.73
    b <- 3.63
  } else if (name == "Sauropoda"){
    a <- 214.44
    b <- 1.46
  } else {
    warning(paste("No known estimation for", name))
    return(NA)
  }
  mass <- a * length ^ b
  return(mass)
}

get_mass_from_length_by_name(6, "Stegosauria")
```

```
[1] 1283.047
```

5. Putting It All Together: Portal Rodents (30 points)

In this question, you will combine choice functions with joins, strings, and plotting. Load the `tidyverse` if you haven't already.

Read `surveys.csv` and `species.csv` (the Portal data we used in Week 4) into R. If you don't have them, you can download them by un-commenting the code below (modify the `destfile` arguments to point to the correct sub-directory).

```
# download.file(url = "https://ndownloader.figshare.com/files/2292172",
#  destfile = "data/surveys.csv")
# download.file(url = "https://ndownloader.figshare.com/files/3299483",
#    destfile = "data/species.csv")
```

```
library(tidyverse)

surveys <- read_csv("data/surveys.csv")
species <- read_csv("data/species.csv")
```

a. Join `surveys` and `species` together so that each survey record has the genus, species, and taxa information. Keep only rodents (`taxa` is "Rodent"). Then, use a `stringr`

function to add a column called `scientific_name` that combines `genus` and `species` with a space between them (e.g., "Dipodomys merriami"). Save the result as an object called `rodents`.

```
# 5a
rodents <- surveys %>%
  left_join(species, by = "species_id") %>%
  filter(taxa == "Rodent") %>%
  mutate(scientific_name = str_c(genus, species, sep = " "))

head(rodents)
```

```
# A tibble: 6 x 13
  record_id month   day  year plot_id species_id sex   hindfoot_length weight
      <dbl> <dbl> <dbl> <dbl>   <dbl> <chr>      <chr>           <dbl>  <dbl>
1         1     7    16  1977       2 NL         M                  32     NA
2         2     7    16  1977       3 NL         M                  33     NA
3         3     7    16  1977       2 DM         F                  37     NA
4         4     7    16  1977       7 DM         M                  36     NA
5         5     7    16  1977       3 DM         M                  35     NA
6         6     7    16  1977       1 PF         M                  14     NA
# i 4 more variables: genus <chr>, species <chr>, taxa <chr>,
#   scientific_name <chr>
```

b. Some individuals could only be identified to genus, so their `species` is "sp." (e.g., "Dipodomys sp."). Use `if_else` and `str_detect` inside a `mutate` to add a column called `id_level` that is "genus only" if `scientific_name` contains "sp." and "species" otherwise. Then use `count()` to show how many records fall into each `id_level`.

Hint: Remember that `.` is a special character in regular expressions! If you don't account for this, some species names (like *Dipodomys spectabilis*) will be counted incorrectly.

```
# 5b
# the . needs to be escaped (or use fixed("sp.")); otherwise "sp." matches
# "sp" followed by any character, such as "spectabilis" or "spilosoma"
rodents <- rodents %>%
  mutate(id_level = if_else(str_detect(scientific_name, "sp\\."),
                            "genus only", "species"))

rodents %>%
  count(id_level)
```

```
# A tibble: 2 x 2
  id_level       n
  <chr>      <int>
1 genus only    70
2 species    34177
```

c. Use `if_else` and the `%in%` operator inside a `mutate` to add a column called `season` that is "warm" if the `month` is April through September (months 4-9) and "cool" otherwise.

```
# 5c
rodents <- rodents %>%
  mutate(season = if_else(month %in% 4:9, "warm", "cool"))
```

d. Use `case_when` inside a `mutate` to add a column called `weight_class` with the following categories:

- "unknown" if `weight` is `NA`
- "small" if `weight` is less than 20
- "medium" if `weight` is greater than or equal to 20 and less than 50
- "large" for everything else (use the `.default` argument)

Save the results of (b), (c), and (d) back into your `rodents` object.

In a sentence or two, answer the following: what would happen to individuals with missing weights if you left out the "unknown" line? Why?

```
# 5d
rodents <- rodents %>%
  mutate(weight_class = case_when(is.na(weight) ~ "unknown",
                                  weight < 20 ~ "small",
                                  weight < 50 ~ "medium",
                                  .default = "large"))

rodents %>%
  count(weight_class)
```

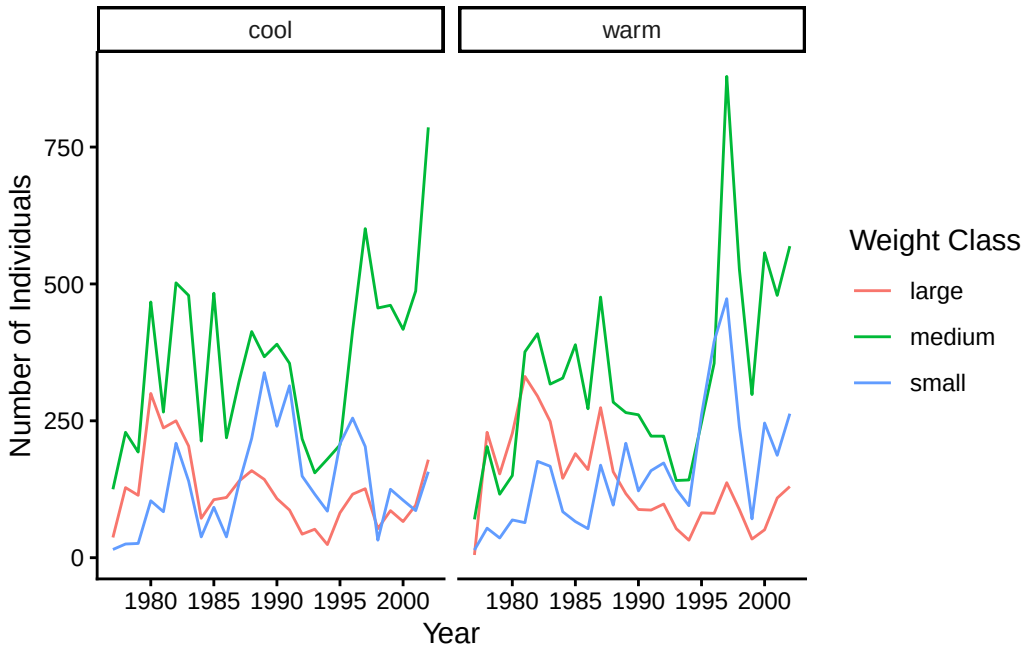
```
# A tibble: 4 x 2
  weight_class     n
  <chr>        <int>
1 large         6721
2 medium       17958
3 small         7604
4 unknown       1964
```

Without the "unknown" line, individuals with missing weights would be classified as "large." `weight < 20` and `weight < 50` both return `NA` when `weight` is `NA`, and `case_when` treats `NA` like `FALSE`, so those rows fall all the way through to the `.default` value.

e. Using `rodents`, remove the individuals with an "unknown" weight class and count the number of individuals in each `year`, `season`, and `weight_class`. Then, make a line plot with `year` on the x-axis, the count on the y-axis, lines colored by `weight_class`, and a separate panel for each `season`. Give your plot informative axis labels and use a theme of your choice.

   In a sentence or two, describe one pattern you see in the plot.

```
# 5e
rodents %>%
  filter(weight_class != "unknown") %>%
  count(year, season, weight_class) %>%
  ggplot(aes(x = year, y = n, color = weight_class)) +
  geom_line() +
  facet_wrap(~ season) +
  labs(x = "Year",
       y = "Number of Individuals",
       color = "Weight Class") +
  theme_classic()
```



Answers will vary. Any accurate description of a pattern in the plot (e.g., which weight class is most common, how counts change over time, or differences between seasons) is acceptable.

6. Challenge: A "Safe" Tidyverse Function (OPTIONAL)

In the lesson, we talked about how `if` statements can control your workflow, such as stopping your code with a helpful error message when something is wrong. Let's try it!

Write a function called `categorize_by_threshold()` that takes three arguments: (1) a data frame, (2) a column in that data frame, and (3) a `threshold` value. (Remember `{ }` from Week 11!)

The function should:

- Use an `if` statement to check whether the column is numeric. If it is not, use `stop()` to end the function with the error message "Column must be numeric."
- Use an `else if` statement to check whether `threshold` is outside the range of values in the column (ignoring `NA`s). If it is, use `stop()` and `paste()` to end the function with an error message that includes the minimum and maximum values of the column, like "Threshold must be between 4 and 280."
- Otherwise, use `mutate()` and `if_else()` to add a new column called `category` that is "above" when the column value is greater than or equal to `threshold` and "below" when it is less than `threshold`.

Hint: `pull(df, {{ col }})` will pull out a column as a vector, which you can then use with functions like `is.numeric()` and `range()`.

```
categorize_by_threshold <- function(df, col, threshold){
  values <- pull(df, {{ col }})

  if (!is.numeric(values)) {
    stop("Column must be numeric.")
  } else if (threshold < min(values, na.rm = TRUE) |
             threshold > max(values, na.rm = TRUE)) {
    stop(paste("Threshold must be between",
               min(values, na.rm = TRUE), "and", max(values, na.rm = TRUE)))
  } else {
    df %>%
      mutate(category = if_else({{ col }} >= threshold, "above", "below"))
  }
}
```

Test your function on the `rodents` data frame from Question 5 four times:

a. with the `weight` column and a threshold of 50
b. with the `hindfoot_length` column and a threshold of 30
c. with the `genus` column and a threshold of 50 (this should give an error)
d. with the `weight` column and a threshold of 500 (this should give an error)

Note: code chunks that produce an error will stop your document from rendering. Add `#| error: true` as the first line of the code chunks for (c) and (d) so the document will render and show the error messages.

```
# 6a
categorize_by_threshold(rodents, weight, 50) %>%
  select(species_id, weight, category)
```

```
# A tibble: 34,247 x 3
   species_id weight category
   <chr>       <dbl> <chr>
 1 NL             NA <NA>
 2 NL             NA <NA>
 3 DM             NA <NA>
 4 DM             NA <NA>
 5 DM             NA <NA>
 6 PF             NA <NA>
 7 PE             NA <NA>
 8 DM             NA <NA>
 9 DM             NA <NA>
10 PF             NA <NA>
# i 34,237 more rows
```

```
# 6b
categorize_by_threshold(rodents, hindfoot_length, 30) %>%
  select(species_id, hindfoot_length, category)
```

```
# A tibble: 34,247 x 3
   species_id hindfoot_length category
   <chr>                <dbl> <chr>
 1 NL                      32 above
 2 NL                      33 above
 3 DM                      37 above
 4 DM                      36 above
 5 DM                      35 above
 6 PF                      14 below
 7 PE                      NA <NA>
```

```
 8 DM                       37 above
 9 DM                       34 above
10 PF                       20 below
# i 34,237 more rows
```

```r
# 6c
categorize_by_threshold(rodents, genus, 50)
```

```
Error in `categorize_by_threshold()`:
! Column must be numeric.
```

```r
# 6d
categorize_by_threshold(rodents, weight, 500)
```

```
Error in `categorize_by_threshold()`:
! Threshold must be between 4 and 280
```